

QUOTA — Product Requirements Document

One line: Anonymous, staked, slashable rate limits for AI-agent traffic. No issuer, no accounts, no tracking.
Track: Monad Metropolis — Trust, Identity & AI Infrastructure (primary output is a primitive, not a consumer product). **Deadline:** 14 Oct 2026, 09:29 IST. Today is 1 Oct. Plan to submit by 13 Oct.

1. Problem

AI agents now generate a large share of API and web traffic. Any free or cheap endpoint (MCP tool servers, search/data APIs, scraped sites) must stop one agent sending millions of requests. Every existing answer breaks something:

Approach	Failure for agents
API keys / accounts	Every call is tied to one identity; a full activity log accumulates. Signup friction.
IP limits	Shared cloud IPs punish honest agents; abusers rotate proxies cheaply.
CAPTCHA	Blocks the legitimate agent too.
Pay-per-call (x402)	Public per-call payments expose the agent's whole history; many endpoints don't want to charge.
Privacy Pass / ARC	Private, but a trusted issuer decides who gets in. A single party captures the trust layer.

Gap: abuse control that is private for the user, enforceable for the service, and controlled by no company.

2. Solution

Agents lock a stake and join a public set. Each request carries a zero-knowledge proof of "I'm a staked member and this is request k of my allowed N this epoch", without revealing which member. Using one request number twice leaks the agent's secret key (Shamir two-point recovery, RLN-v2). Anyone with the secret can slash the stake. The slash transaction travels through BTX so a block leader cannot copy the secret and steal the reward. A passkey tap gates all stake custody actions.

3. Users and adopters

Persona	Need	What they do with QUOTA
Service operator (MCP host, search/data API, anti-scraping site)	Limit abuse without building accounts or bot detection	Add one middleware line; set N per epoch and accepted trees; receives slash rewards
Agent operator (human or team running agents)	Let agents call many services without leaving a trail	Stake once via passkey; agent signs requests through the SDK
Agent framework / SDK author	"Stake once, call anywhere" default	Embed <code>@quota/client</code>

Launch adopters to pursue (must name at least one committed integrator before submission): MCP server hosts, free data/search APIs, Monad-ecosystem endpoints. Public RPC is explicitly *not* a target (per-request proof latency).

4. Goals and non-goals

Goals

1. A working primitive on Monad testnet: contracts, SDKs, a drop-in middleware, a reference MCP server, and a console.
2. Correct passkey (WebAuthn/P-256) custody verified on-chain via the P256 precompile.
3. A slash path that is demonstrably safe against copy-and-steal front-running (BTX, with a documented commit-reveal fallback).
4. Developer experience: a service operator integrates in under 10 minutes; an agent developer in 5 lines.
5. Honest sponsor integrations where each removes real capability if deleted (see `00-bounty-validation.md`).

Non-goals

- Not proof of personhood. A passkey is not a human; the **stake** provides sybil cost.
- Not a global rate budget. Quotas are per server.
- Not a payments system. No per-call payments.
- Not low-latency infrastructure for public RPC.
- No new identity registry. ERC-8004 linkage is optional, and opt-in only.

5. Core concepts

- **Operator:** the human/org with a passkey. Public at enrollment.
- **Leaf:** `rateCommitment = Poseidon(Poseidon(a0), limit)` in the Merkle tree. `a0` is the agent's secret, `limit` is its per-epoch allowance (linear in stake, so splitting stake gains nothing).
- **Epoch:** a time window (e.g., 1 hour). **Message id** `k` runs `0..limit-1`.
- **External nullifier:** `H(serverId, epoch)`, so quotas are per server and shares from different servers never collide.
- **Tree:** an incremental Merkle tree of leaves. Trees: **Open** (default), **Screened** (Nansen attestation), **Compliant** (Cleanverse CVI; stake as CVA). Services choose which they accept.
- **Slash:** `slash(a0, limit, receiver)` removes the leaf and pays `SLASH_SHARE` of the stake to `receiver`, remainder burned/treasury.

6. Functional requirements

6.1 Contracts (Solidity, Foundry)

ID	Requirement
C1	<code>QuotaRegistry</code> : incremental Poseidon Merkle tree(s); <code>enroll</code> , <code>topUp</code> , <code>requestUnstake</code> / <code>unstake</code> , <code>slash</code> .
C2	<code>PasskeyAuth</code> library: full WebAuthn assertion verification via P256VERIFY at <code>0x100</code> . Checks: <code>clientDataJSON.type == "webauthn.get"</code> , challenge match, origin allowlist, <code>rpIdHash</code> , UP and UV flags, low-s normalization, per-operator nonce.

ID	Requirement
C3	Passkey registered once per operator (<code>registerPasskey(x, y)</code>), then required for: unstake, limit/tier change, withdrawal destination . The signed payload binds (chainId, registry, operator, action, params, nonce) .
C4	<code>enroll(rateCommitment, limit, treeId)</code> payable; $\text{stake} \geq \text{limit} \times \text{UNIT}$. Enrollment authorization requires the passkey assertion for the operator.
C5	<code>slash(a0, limit, receiver)</code> : recompute leaf from <code>a0</code> , verify membership, delete leaf, pay receiver. Must be called through <code>SubmitPath</code> (see 6.4).
C6	Recent-roots window (last R roots) so removals take effect within a documented TTL.
C7	Optional hook: emit ERC-8004 Validation Registry record when an opted-in agent is slashed. Default off.
C8	Tree policy: Open = anyone; Screened = enrollment needs attester signature; Compliant = stake token is CVA and CVI check passes (Phase 6/0 gate).

6.2 Proofs (RLN-v2)

ID	Requirement
Z1	Reuse PSE RLN-v2 circuits (Circom/Groth16); no custom circuit. Verifier key and params pinned and checksummed.
Z2	Prover in TS (Node + browser/WASM). Target proof time recorded in Phase 2 (budget ≤ 2 s on a laptop; report the real number).
Z3	Verifier in TS (off-chain, ms). On-chain Groth16 verifier optional, used only by tests/dispute.
Z4	Server stores (externalNullifier, nullifier) $\rightarrow$ (x, y) ; on a repeat nullifier with different x , recover <code>a0</code> and enqueue slash.

6.3 SDKs and middleware

ID	Requirement
S1	<code>@quota/client</code> : <code>signRequest(serverId, payloadHash) $\rightarrow$ headers</code> , tracks <code>k</code> per epoch, refuses to exceed local limit unless <code>allowOveruse</code> (used for the violation demo).
S2	<code>@quota/server</code> : middleware (Express/Hono + MCP server wrapper) that verifies proofs, maintains the nullifier store, exposes <code>onViolation</code> .
S3	<code>@quota/slasher</code> : watches the store, submits <code>slash</code> via <code>SubmitPath</code> using a pluggable wallet adapter.
S4	Wallet adapters (one interface): <code>PrivyAgentWallet</code> , <code>DynamicServiceWallet</code> , <code>LocalKeyWallet</code> . Sign-only; broadcast is ours.
S5	Docs: quickstart for each persona, API reference, threat model, and a one-page "why not roll your own".

6.4 SubmitPath (BTX)

ID	Requirement
B1	One interface <code>SubmitPath.send(tx)</code> with <code>btx</code> and <code>commitReveal</code> implementations.

ID	Requirement
B2	<code>commitReveal : commitSlash(H(a0, limit, receiver, salt))</code> , then <code>revealSlash(...)</code> after ≥ 1 block. This is the guaranteed fallback.
B3	<code>btx</code> : use whatever the Monad BTX interface exposes on testnet. Phase 0 gate: confirm it exists and whether it hides target + calldata or only payload. If unavailable, ship commit–reveal and state it plainly. Never present a self-made contract as BTX.

6.5 Wallet integrations

ID	Requirement
W1	Privy (agent side): operator logs in; backend creates a server wallet owned by the operator's user, with our runtime authorization key as additional signer and override policies (allowlist: QuotaRegistry only; value cap).
W2	Agent secret <code>a0</code> derived from a Privy wallet signature over a fixed domain string; recoverable with the wallet.
W3	Dynamic (service side): service operator logs in; embedded wallet receives rewards; delegated access lets the slasher sign <code>slash</code> without a prompt. Fallback: Dynamic server wallet.
W4	Both providers used in sign-only mode so transactions go through <code>SubmitPath</code> .

6.6 Trust tiers (conditional)

ID	Requirement
T1	Screener (Nansen): service that profiles a new operator wallet (labels, related wallets, funding source, counterparties); if clean, signs an attestation accepted by the Screened tree. Anyone can run one; trees list accepted attester keys.
T2	Recidivism rule: wallets related to a previously slashed wallet fail screening, or must post $2\times$ stake.
T3	Compliant tree (Cleanverse): stake token is a CVA; staking and slash payouts only between CVI-verified wallets; CVI checked on-chain before any CVA movement.

6.7 Demo and console

ID	Requirement
D1	Qwen "Scout" agent: tool-calling loop (<code>quota_status</code> , <code>call_tool</code> , <code>topup_stake</code> , <code>switch_server</code>) that plans multi-server research within quota.
D2	Reference MCP server protected by QUOTA (e.g., web-search stub with real data).
D3	Violation mode: a script that deliberately exceeds quota, leaking its secret on screen, and the slash landing on-chain.
D4	Operator console: Privy login, create agent, passkey enroll, stake/top-up/unstake.
D5	Service console: Dynamic login, register server, set N and accepted trees, live request/violation feed, slash history, rewards. Nansen panels if Phase 6 ships.

7. Non-functional requirements and security

- **Passkey correctness** is the Technical Execution core: challenge binding, replay protection (nonce), origin/rpld checks, low-s, signature malleability. Dedicated test vectors, including negative tests.
- **No leaked secrets:** `a0` never logged or stored in plaintext; it exists only in agent memory and on-chain at slash time.
- **Slash race:** unprotected `slash(a0, ...)` can be copied. All production paths use BTX or commit–reveal. Test: a searcher bot tries to copy-and-steal and must fail.
- **Per-server quotas:** document that aggregate exposure is N per server.
- **Stake must exceed abuse value:** service chooses a minimum tier. Document guidance.
- **Passkey loss:** a lost passkey means a stuck stake. Document recovery (second passkey registration with delay) as a stretch.
- **Privacy limits:** operator is public at enrollment. Anonymity set = all leaves in the accepted trees at the time of the request. Say so.
- **Latency budget:** proof generation measured and reported honestly, not claimed.

8. Success metrics

Metric	Target by submission
Live testnet deployment with verified contracts	yes
End-to-end demo: enroll → 50+ anonymous requests → violation → BTX/commit–reveal slash → reward	yes
Named external integrator (a team's MCP server or API running the middleware)	≥ 1
Service integration time (measured, new dev)	≤ 10 min
Contract test coverage (passkey + slash paths)	all negative cases listed in 7
Sponsor integrations passing the "remove it and it breaks" test	Privy, Qwen (+ Dynamic if gate passes)

9. Risks

Risk	Impact	Mitigation
BTX not usable on testnet	Slash "front-running protection" falls back to commit–reveal	Phase 0 gate; ship fallback, state it plainly
Poseidon tree insert gas / Monad limits	Enrollment too costly	Measure Phase 1; fall back to off-chain tree + posted root with fraud check
Privy signature not deterministic	Secret derivation breaks	Phase 0 test; fallback encrypted blob
Dynamic delegated access awkward on Monad	Service wallet role fails	Phase 0 spike + cut rule
Proof latency too high	Demo feels slow	Measure early; pre-warm prover; smaller depth
No service wants anonymity (sellers prefer identifying clients)	Weak market score	Target free/unpaid endpoints; secure a named integrator early

Risk	Impact	Mitigation
Scope (7 integrations in 13 days)	Missed deadline	Cut order: Cleanverse → Nansen → Dynamic; protocol and Privy/Qwen never cut

10. Positioning (for pitch and judging)

- **Lead line:** "ARC without the issuer": Apple/Cloudflare-style private rate limits, but permissionless and economically enforced on-chain.
- **Track fit:** trust and privacy are the core; data-ownership angle = "your agent's activity history is your data, and today access costs you that data."
- **Why not roll your own:** RLN circuits are audited and reused; passkey custody, slashing safety, SDKs, and wallet adapters are done once.